COS 214 - P3

Rei Piater-Boswell (u25678592)
Leanne van der Horst (u24566935)
Declan (u -)

Event name & description:

UML class diagram:

Participant mapping:

Composite Participants:

EventComponent: Component
StageZone, FoodCourt, IndieGaming and Arts: Composites
GamingBooth, Paramedics, Cleaners, ESports, DemoZone, MeetAndGreet, and ProductBooth: Leaves

Observer Participants:

Subject: Abstract Subject
EventControl and Composites: Concrete Subject
Observer: Abstract Observer
EventComponent, Leaves, and Composites: Concrete Observer
The reason for the Composites being Observers as well as Subjects is that they may be part of a hierarchy of a different zone and need to keep track of the change in state in said hierarchy. The reason for them also being Subjects is that their states can change and their observers, the leaves or other composites, need to keep track of the changes and act accordingly. This is seen in the ESports leaf that is observing its parent Stage composite and can, for example, cause all the people that are sitting in the seating area to leave if an EVACUATE message is sent, or even if the state of the Stage composite is changed to close.

Design rationale:

a)
b) The Observer pattern is required for the functionality of the program, because there could be a lot of state changes that happened to a subject and could cause some hard coded calls to not properly function since they may not be able to access certain data that could have changed. This is also because it would allow for the program to run more efficiently since the calls are being handled by an external object rather than the object itself.

c) The composites would be the objects that can contain the event objects as they are the leaves to the Composite pattern.
d)
e) The reason why an object can be both an observer and a subject without being a misuse of either pattern is because an object can be owned by a parent whilst being a parent to children objects itself. This allows for the object to observe its parent and act according to the state changes and even send those changes down to its children to allow for them to react to something that is affecting their parent.
Our Observer pattern uses push to send notifications down to observers<finish later>

Composite object diagram:

Event rules and original features:

OPEN: Sets a zone or individual objects, such as a booth, to be seen as open. If it is already open then nothing happens. For example, this causes it a stage to be able to receive more people adding to its capacity and be interacted with.
CLOSE: Sets a zone or individual objects, such as a booth, to be seen as closed causing it to not be active, for example a stage loses its functionality and has no capacity.
MAX_CAPACITY: if a zone has a capacity and the current capacity of the zone exceeds its set max capacity then the zone would no longer accept more people and “kick” some people out to go down to the max capacity. However, if a zone does not keep track of its capacity then it would do nothing different - continue as normal.
EVACUATE: Sets all zones to be closed, clears everything’s capacities causing everything to be seen as closed and cleared of people.
STAGE_EVENT: If the zone is a Stage Zone then it would cause all the different stages to combine into one zone, adding their max capacities together and acting as one stage. If a zone is not a Stage Zone then it would continue as normal since it is not a stage.

Sequence Diagrams:

SD1

SD2

SD3

SD4

Doxygen evidence :

Github Repo link:

https://github.com/Lenn-lolz/COS214_P3.git

GitHub workflow reflection:

Team contribution statement:

All three of the members contributed to the creation of the UML's design for the EventFlow project.